

Algorytmy Równoległe - Laboratorium 7

Adam Staniszewski

10 grudnia 2024

1 Implementacja sekwencyjna

Dla porównania z innymi metodami, przygotowałem implementację sekwencyjną.

Stałe

```
G = 6.67430e-11
EPS = 1e-10
MASS_SCALAR = 1e30
```

Obliczanie przyspieszenia

```
def compute_forces(stars: list[dict]) -> list[dict]:
    for star in stars:
        star['force'] = np.zeros(3)
        star['acceleration'] = np.zeros(3)

    for i, star_i in enumerate(stars):
        for j, star_j in enumerate(stars):
            if i != j:
                r_vec = star_j['position'] - star_i['position']
                r_mag = np.linalg.norm(r_vec)

                force = G * star_i['mass'] * star_j['mass']
                * r_vec / (r_mag**3 + EPS)
                star_i['force'] += force

        star_i['acceleration'] = star_i['force'] / star_i['mass']

    return stars
```

Wyniki

Dla pięciu gwiazd o danej masie i położeniu:

```
stars = [
    {'position': np.array([1.0, 0.0]), 'mass': MASS_SCALAR},
    {'position': np.array([1.0, 2.0]), 'mass': MASS_SCALAR},
    {'position': np.array([2.0, 4.0]), 'mass': MASS_SCALAR},
    {'position': np.array([3.0, 6.0]), 'mass': MASS_SCALAR},
    {'position': np.array([4.0, 8.0]), 'mass': MASS_SCALAR},
]
```

Gwiazda	Przyspieszenie X	Przyspieszenie Y
1	1.80088671e+18	2.29336100e+19
2	8.12539152e+18	-4.34966964e+17
3	5.40209720e+17	-8.23998822e+17
4	-2.02006859e+18	-4.56778693e+18
5	-8.44641935e+18	-1.71068573e+19

Tabela 1: Przyspieszenia gwiazd dla implementacji sekwencyjnej.

W celu wyliczenia efektywności, przyspieszenia i metryki Karpa-Flatta przeprowadziłem również eksperyment z użyciem **4000 gwiazd**, które zostały wygenerowane przez następujący kod:

```
def generate_stars(num_stars):
    stars = []
    for _ in range(num_stars):
        position = np.random.rand(2) * 10
        mass = MASS_SCALAR
        stars.append({'position': position, 'mass': mass})
    return stars
```

Algorytm sekwencyjny wykonał wszystkie obliczenia w **96.46** sekundy.

2 Zadanie 1

```
def task_1(stars, rank, size):
    N = len(stars)
    stars_per_process = N // size
    local_stars = stars[rank * stars_per_process: (rank + 1)
                        * stars_per_process]

    transfer_buffer = np.copy(local_stars)
    local_acc = np.zeros_like(local_stars)

    comm = MPI.COMM_WORLD

    for _ in range(size - 1):
        left_rank = (rank - 1) % size
```

```

right_rank = (rank + 1) % size

comm.send(transfer_buffer, dest=left_rank)
transfer_buffer = comm.recv(source=right_rank)

for i, star_i in enumerate(local_stars):
    m_i = star_i['mass']
    r_i = star_i['position']

    for star_j in transfer_buffer:
        r_j = star_j['position']
        m_j = star_j['mass']

        r_vec = r_j - r_i
        r_mag = np.linalg.norm(r_vec)
        force = G * m_i * m_j * r_vec / (r_mag**3 + EPS)
        local_acc[i] += force / m_i

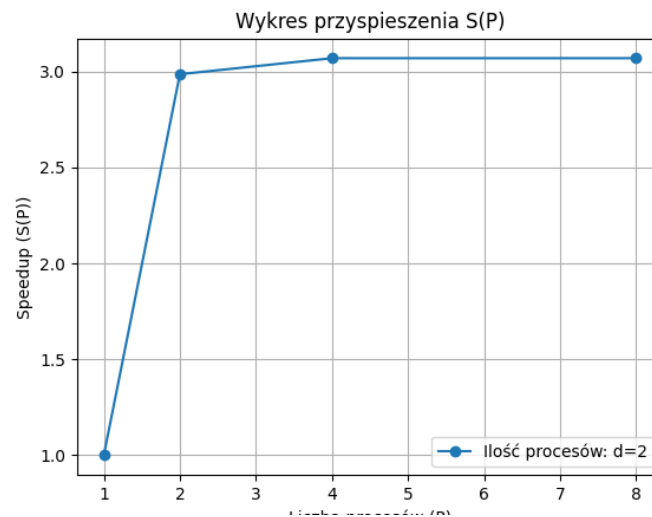
return local_acc

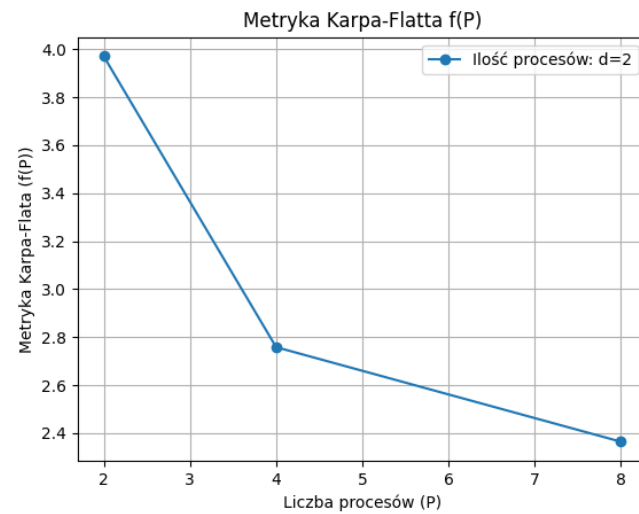
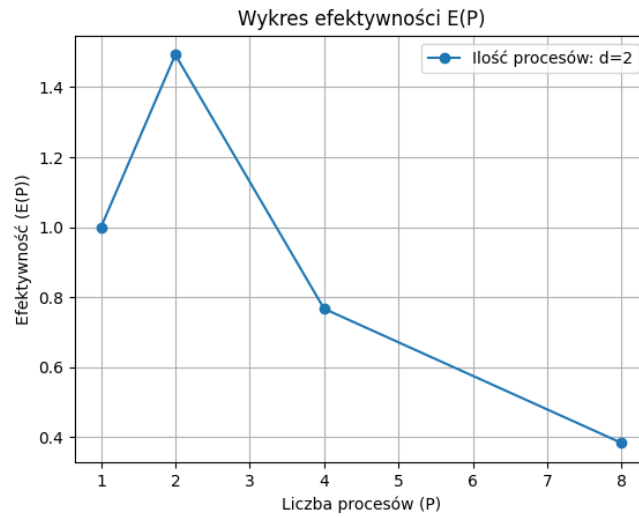
```

Dla tego samego zestawu gwiazd, co w poprzednim zadaniu, otrzymałem zbliżone wyniki.

Ilość rdzeni	Czas wykonania (s)
Implementacja sekwencyjna	96.46
2	32.31
4	31.43
8	35.18

Tabela 2: Czas wykonania algorytmu dla 4000 gwiazd przy różnej liczbie rdzeni.





3 Zadanie 2

```
def task_2(stars, rank, size):
    N = len(stars)
    stars_per_process = N // size
    local_stars = stars[rank * stars_per_process: (rank + 1)
                       * stars_per_process]

    transfer_buffer = np.copy(local_stars)
```

```

local_acc = np.zeros_like(local_stars)

comm = MPI.COMM_WORLD

for _ in range(size // 2):
    left_rank = (rank - 1) % size
    right_rank = (rank + 1) % size

    comm.send(transfer_buffer, dest=left_rank)
    transfer_buffer = comm.recv(source=right_rank)

    for i, star_i in enumerate(local_stars):
        m_i = star_i['mass']
        r_i = star_i['position']

        for star_j in transfer_buffer:
            r_j = star_j['position']
            m_j = star_j['mass']

            r_vec = r_j - r_i
            r_mag = np.linalg.norm(r_vec)
            force = G * m_i * m_j * r_vec / (r_mag**3 + EPS)
            local_acc[i] += force / m_i

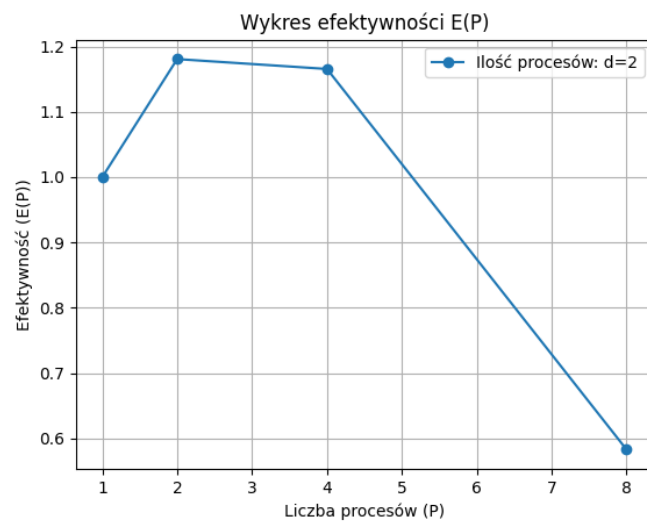
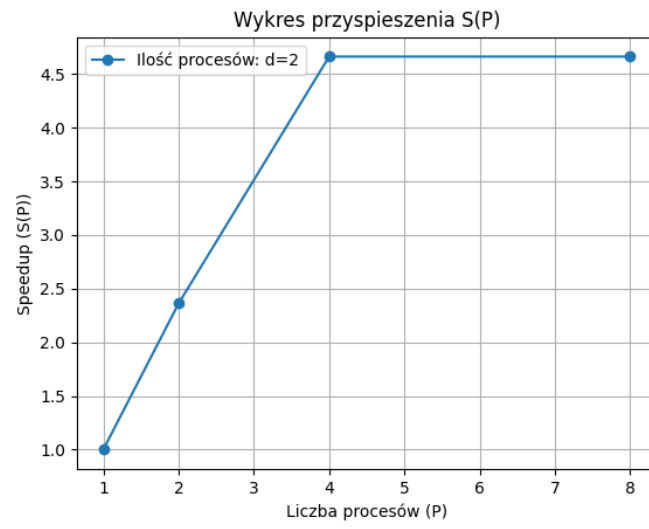
    transfer_buffer = local_stars.copy()

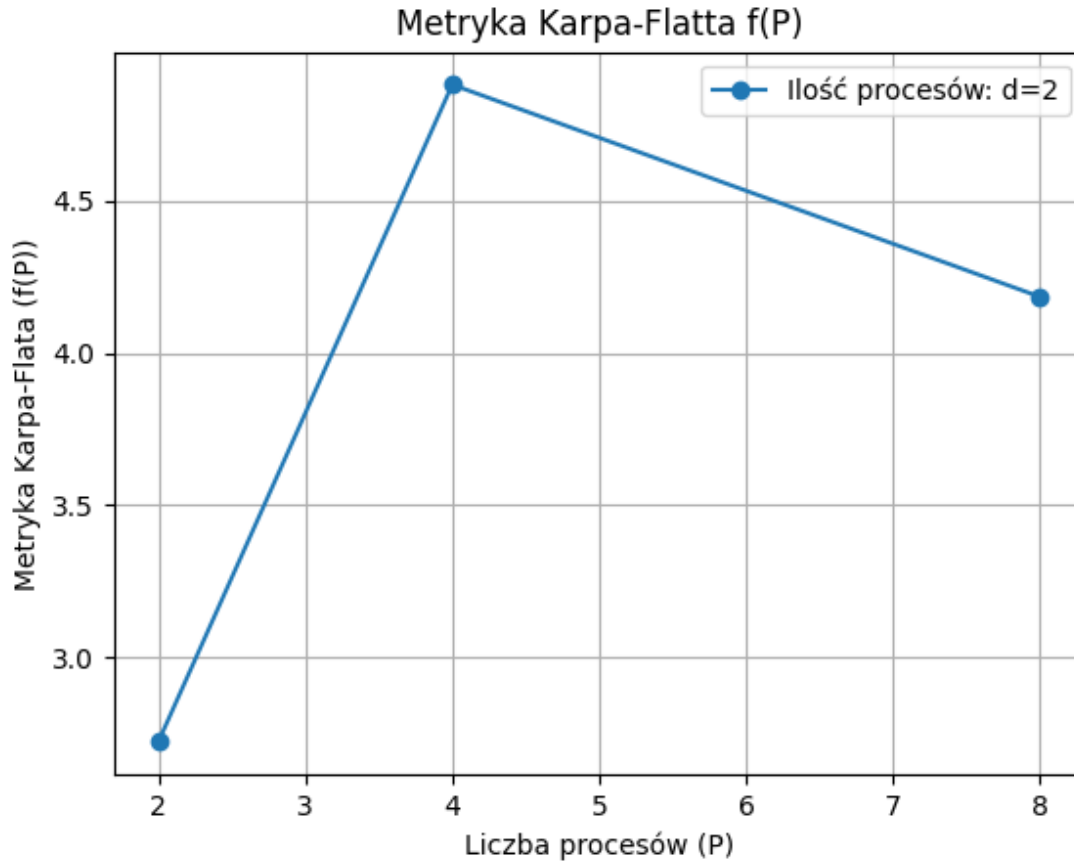
return local_acc

```

Ilość rdzeni	Czas wykonania (s)
Implementacja sekwencyjna	96.46
2	40.85
4	20.69
8	19.27

Tabela 3: Czas wykonania algorytmu dla 4000 gwiazd przy różnej liczbie rdzeni.





4 Zadanie 3

W tym zadaniu wykonujemy znacznie więcej obliczeń, ponieważ korzystamy zarówno z algorytmu z **zadania 2**, jak i dokonujemy obliczeń przy użyciu metody Eulera. W związku z tym zmniejszyłem liczbę gwiazd do **1000**.

Korzystając z metody Eulera, aktualizujemy położenie:

$$\mathbf{r}(t + \Delta t) = \mathbf{r}(t) + \mathbf{v}(t) \cdot \Delta t$$

i prędkość:

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \mathbf{a}(t) \cdot \Delta t$$

```
def task_3(stars, rank, size, time_steps, dt):
    N = len(stars)
    stars_per_process = N // size
    local_stars = stars[rank * stars_per_process: (rank + 1) * stars_per_process]
```

```

* stars_per_process]

for star in local_stars:
    star['velocity'] = np.zeros(2)

comm = MPI.COMM_WORLD

for _ in range(time_steps):
    local_acc = task_2(stars, rank, size)

    for i, star in enumerate(local_stars):
        # Pozycja
        star['position'] += star['velocity'] * dt
        # Prdko
        star['velocity'] += local_acc[i] * dt

    all_stars = comm.allgather(local_stars)
    stars = [star for sublist in all_stars for star in sublist]

return local_stars

```

Ilość rdzeni	Czas wykonania (s)
Implementacja sekwencyjna	779.56
2	244.64
4	183.43
8	157.95

Tabela 4: Czas wykonania algorytmu dla 1000 gwiazd przy różnej liczbie rdzeni.

```

position,mass,velocity
[-4.42521071e+33  4.62379473e+31],1e+30,[-4.49259970e+26  4.69420785e+24]
[ 6.10195254e+33 -1.07909614e+33],1e+30,[ 6.19487568e+26 -1.09552908e+26]
[ 6.48448047e+32 -3.10381954e+33],1e+30,[ 6.58322891e+25 -3.15108583e+26]
[-2.27316337e+33 -3.60782768e+33],1e+30,[-2.30778007e+26 -3.66276922e+26]
[ 2.84317864e+33 -1.17144343e+33],1e+30,[ 2.88647577e+26 -1.18928267e+26]
[-2.46132216e+31  3.01733450e+33],1e+30,[-2.49880422e+24  3.06328375e+26]
[ 2.97619056e+34 -1.08554503e+34],1e+30,[ 3.02151326e+27 -1.10207617e+27]
[ 7.23598937e+33 -1.59692995e+33],1e+30,[ 7.34618210e+26 -1.62124868e+26]
[9.61752209e+32  2.52518287e+32],1e+30,[9.76398181e+25  2.56363743e+25]

```

Rysunek 1: Wyniki dla kilku przykładowych gwiazd

